

Introduction to Backpropagation

Backpropagation computes the gradient of the loss function with respect to every weight in the network by applying the chain rule backwards through each layer in turn. The algorithm is what makes training a deep network tractable, because it reuses the partial results of one layer when computing the gradients of the next. Without it, each weight would need its own forward evaluation.

The algorithm proceeds in two passes over the network. The forward pass computes the activation of every unit and stores it, because the backward pass will need those stored values to evaluate the local derivatives. Memory therefore grows with the depth of the network and the size of the batch being processed.

The backward pass begins at the loss and works towards the input. At each layer it receives the gradient of the loss with respect to that layer's output, and produces two things from it: the gradient with respect to the layer's parameters, which the optimiser will use, and the gradient with respect to the layer's input, which is what the preceding layer receives in its turn.

Numerical stability deserves attention at this point. Gradients that are repeatedly multiplied by values below one shrink towards zero as they travel backwards, and the layers nearest the input stop learning altogether. Multiplying by values above one has the opposite effect and the gradients diverge. Both failures are properties of the composition rather than of any single layer.

Practical implementations address this with careful initialisation, normalisation between layers, and architectural choices that give gradients a shorter path back to the input. None of these change the algorithm itself. They change the numbers it operates on so that the arithmetic stays within a usable range.

The Backward Pass in Detail

Gradients flow backwards from the output layer towards the input. Each layer receives the gradient of the layer above it and applies the local derivative of its own operation before passing the result along. The composition of these local derivatives is exactly what the chain rule describes.

Weight updates follow the optimiser's rule rather than the gradient directly. Stochastic gradient descent subtracts a scaled gradient, while adaptive methods maintain per-parameter statistics and scale each update by them. The gradient computation is identical in both cases; only what is done with the result differs.